

REST API

Complete notes · HTTP methods, status codes, design best practices and interview questions

1. Understanding REST APIs

REST (Representational State Transfer) is an architectural style for designing networked applications. It relies on stateless, client-server communication over HTTP using standard methods and status codes. RESTful APIs are designed around **resources** — which can be anything from users and products to documents.

Key Concepts

1. **Resources:** Everything that can be accessed via a RESTful API is considered a resource. Each resource has a unique identifier (URI).
2. **Representations:** Resources are transferred in some representation such as JSON or XML.
3. **Stateless Communication:** Each request from client to server contains all the information needed. The server does not store any state about the client session.
4. **HTTP Methods:** REST APIs use standard HTTP methods to perform actions on resources.
5. **HTTP Status Codes:** Servers use HTTP status codes to indicate the outcome of a client's request.

2. HTTP Methods

An HTTP method defines the action you want to perform on a resource. These are the most commonly used ones:

Method	Description	Idempotent?	Safe?
GET	Retrieves a resource or a list of resources. Should not modify data on the server.	Yes	Yes
POST	Creates a new resource. The request body contains the data for the new resource.	No	No
PUT	Updates a resource by replacing it entirely. Requires the complete new representation in the request body.	Yes	No
PATCH	Updates a resource by partially modifying it. Requires only the modified fields in the request body.	No*	No
DELETE	Deletes a resource.	Yes	No

* PATCH is **not guaranteed** to be idempotent. It can be written in an idempotent way (setting a field to a fixed value), but a patch like "increase price by 10" changes the result every time it runs.

Important Notes on Methods

Idempotence

An idempotent method produces the same end result whether it is called once or many times with the same request. GET , PUT and DELETE are idempotent. POST is not — calling it twice creates two resources.

Safe Methods

A safe method must not modify data on the server; it is read-only. `GET`, `HEAD` and `OPTIONS` are safe methods. Every safe method is automatically idempotent, but the reverse is not true.

3. HTTP Status Codes

Status codes are three-digit numbers the server uses to indicate the outcome of a client's request. They are grouped into five classes:

Status Code Range	Meaning
1xx — Informational	The request was received and the process is continuing.
2xx — Success	The request was successfully received, understood and accepted.
3xx — Redirection	Further action needs to be taken by the client to fulfil the request.
4xx — Client Error	The client sent a request with invalid syntax, or one that could not be fulfilled.
5xx — Server Error	The server failed to fulfil an apparently valid request. These usually indicate problems with the server or backend services.

Common Status Codes

Code	Category	Name / Description	Use Case
200	Success	OK — the request was successful.	A successful GET request
201	Success	Created — a new resource has been created.	A successful POST request
204	Success	No Content — the request succeeded but there is no body to return.	DELETE request when the resource is removed successfully
301	Redirection	Moved Permanently — the resource has moved to a new URL.	Redirecting an old URL to a new one
302	Redirection	Found — the resource is temporarily at a different URL.	Temporary redirect to a different URL
304	Redirection	Not Modified — the resource has not changed since the last request.	Conditional GET to prevent unnecessary transfer
400	Client Error	Bad Request — the request was malformed or invalid.	Invalid request body or missing parameters
401	Client Error	Unauthorized — the user is not authenticated.	Accessing a protected resource without a valid token
403	Client Error	Forbidden — the user is authenticated but not authorized.	Permission denied for this role
404	Client Error	Not Found — the resource does not exist.	Invalid URL or deleted record

Code	Category	Name / Description	Use Case
405	Client Error	Method Not Allowed — the HTTP method is not supported on this route.	POST on a read-only endpoint
409	Client Error	Conflict — the request conflicts with the current state of the resource.	Registering an email that already exists
422	Client Error	Unprocessable Entity — the syntax is valid but the data fails validation.	Password too short, invalid date format
500	Server Error	Internal Server Error — something went wrong on the server.	Generic server-side failure
501	Server Error	Not Implemented — the feature is not supported by the server.	Unsupported operation
503	Server Error	Service Unavailable — the server is temporarily unavailable.	Server overload or maintenance window

4. RESTful API Design Best Practices

- Use nouns to represent resources — `/users` , `/products` — never verbs like `/getUsers` .
- Use plural nouns for collections — `/users` , and `/users/:id` for a single item.
- Use HTTP methods according to their semantics (`GET` for read, `POST` for create, and so on).
- Use status codes appropriately to convey the outcome of the request.
- Keep your APIs consistent and predictable across every route.
- Design stateless APIs that do not rely on session storage on the server.
- Version your API (`/api/v1/users`) so future changes do not break existing clients.

5. Interview Questions and Answers

Q1. What is a REST API?

REST (Representational State Transfer) is an architectural style for designing networked applications. It relies on stateless client-server communication using standard HTTP methods and status codes. RESTful APIs are designed around resources, which can be accessed using methods like `GET` , `POST` , `PUT` , `PATCH` and `DELETE` .

Q2. What are the common HTTP methods used in REST APIs, and what do they mean?

- `GET` — retrieve a resource or a list of resources.
- `POST` — create a new resource.
- `PUT` — update a resource by replacing it with new data.
- `PATCH` — update a resource by partially modifying it.
- `DELETE` — delete a resource.

Q3. Explain the difference between PUT and PATCH.

- `PUT` — used to completely replace a resource. It expects the entire resource representation in the request body. Any field you leave out gets wiped.
- `PATCH` — used to partially modify a resource. It only requires the fields that need to be updated in the request body.

Q4. What are HTTP status codes, and why are they important in REST APIs?

HTTP status codes are three-digit numbers the server sends to the client to indicate the outcome of a request. They are important because they let the client understand whether a request succeeded, failed, or needs further action — without having to parse the response body. They also help a lot while troubleshooting API issues.

Q5. Give some examples of common HTTP status codes and when they are used.

- `200 OK` — the request was successful (for example, retrieving a resource).
- `201 Created` — a new resource was successfully created (for example, a POST request).
- `400 Bad Request` — the request was invalid.
- `401 Unauthorized` — the client needs authentication (API key or credentials).
- `403 Forbidden` — the client is authenticated but does not have permission to access the resource.
- `404 Not Found` — the resource does not exist.
- `500 Internal Server Error` — a server-side error occurred.

Q6. What does it mean for a method to be idempotent? Which HTTP methods are idempotent?

An idempotent method produces the same result on the server regardless of how many times it is called with the same request. `GET`, `PUT` and `DELETE` are idempotent. `POST` is not, because each call creates a new resource. `PATCH` is not guaranteed to be idempotent — it depends entirely on how the patch operation is written.

Q7. What does it mean for a method to be safe? Which HTTP methods are safe?

A safe method must not modify any data on the server — it is purely read-only. `GET`, `HEAD` and `OPTIONS` are safe methods.

Q8. If an API operation fails, should the API return an error status code?

Yes. An API should always return a proper error status code along with a descriptive message. For example:

- Use `400 Bad Request` for invalid request data.
- Use `404 Not Found` if the resource is not available.
- Use `500 Internal Server Error` when an unexpected error happens on the server.

Q9. What are some best practices for designing RESTful APIs?

- Use nouns to represent resources and pluralise collections.
- Use HTTP methods according to their semantics.
- Use status codes appropriately to convey the outcome of the request.
- Keep your APIs consistent and predictable.
- Design stateless APIs.

Q10. How do you handle errors in RESTful APIs?

- Use proper HTTP status codes to indicate the type of error.
- Include a descriptive error message in the response body, usually in JSON format.
- Keep the error shape consistent across the whole API so the frontend can handle it in one place.
- Log errors on the server side for debugging purposes.